

2463. Minimum Total Distance Traveled

Solved 

Hard

 Topics

 Companies

 Hint

There are some robots and factories on the X-axis. You are given an integer array `robot` where `robot[i]` is the position of the i^{th} robot. You are also given a 2D integer array `factory` where `factory[j] = [positionj, limitj]` indicates that `positionj` is the position of the j^{th} factory and that the j^{th} factory can repair at most `limitj` robots.

The positions of each robot are **unique**. The positions of each factory are also **unique**. Note that a robot can be **in the same position** as a factory initially.

All the robots are initially broken; they keep moving in one direction. The direction could be the negative or the positive direction of the X-axis. When a robot reaches a factory that did not reach its limit, the factory repairs the robot, and it stops moving.

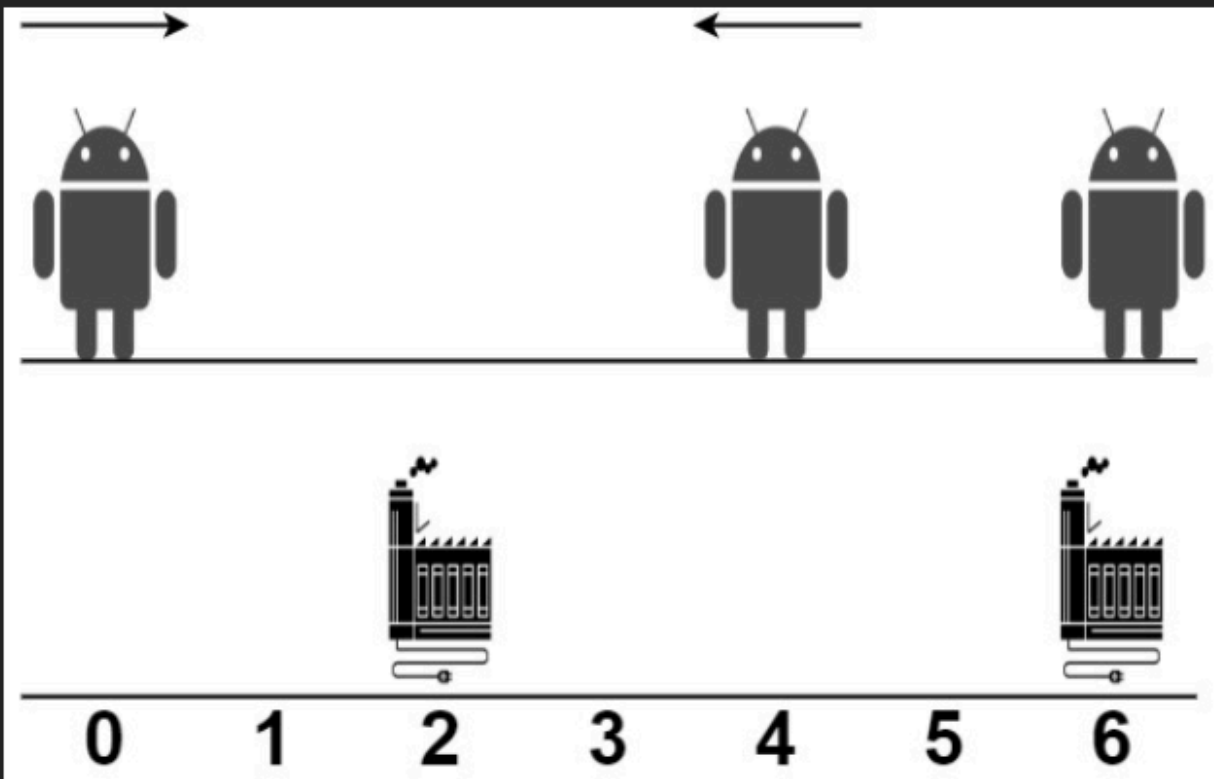
At any moment, you can set the initial direction of moving for **some** robot. Your target is to minimize the total distance traveled by all the robots.

Return *the minimum total distance traveled by all the robots*. The test cases are generated such that all the robots can be repaired.

Note that

- All robots move at the same speed.
- If two robots move in the same direction, they will never collide.
- If two robots move in opposite directions and they meet at some point, they do not collide. They cross each other.
- If a robot passes by a factory that reached its limits, it crosses it as if it does not exist.
- If the robot moved from a position `x` to a position `y`, the distance it moved is `|y - x|`.

Example 1:



Input: robot = [0,4,6], factory = [[2,2],[6,2]]

Output: 4

Explanation: As shown in the figure:

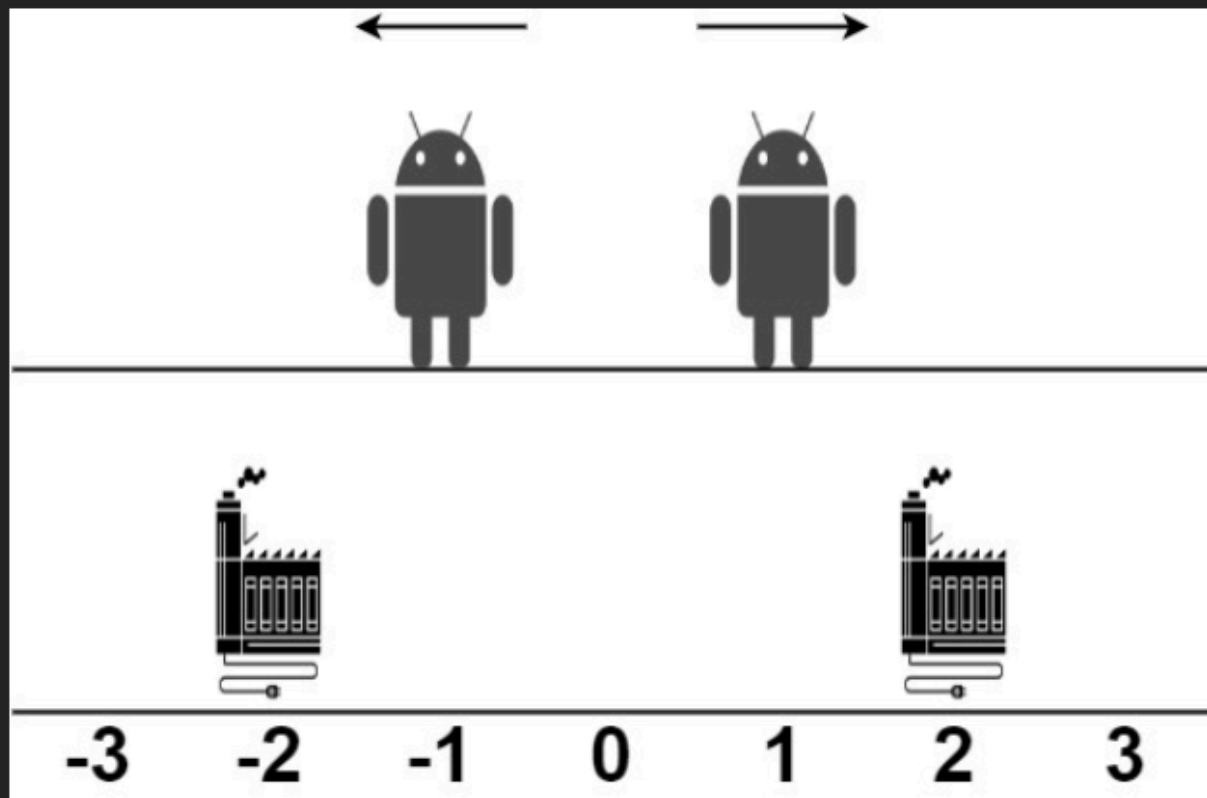
- The first robot at position 0 moves in the positive direction. It will be repaired at the first factory.
- The second robot at position 4 moves in the negative direction. It will be repaired at the first factory.
- The third robot at position 6 will be repaired at the second factory. It does not need to move.

The limit of the first factory is 2, and it fixed 2 robots.

The limit of the second factory is 2, and it fixed 1 robot.

The total distance is $|2 - 0| + |2 - 4| + |6 - 6| = 4$. It can be shown that we cannot achieve a better total distance than 4.

Example 2:



Input: robot = [1,-1], factory = [[-2,1],[2,1]]

Output: 2

Explanation: As shown in the figure:

- The first robot at position 1 moves in the positive direction. It will be repaired at the second factory.
- The second robot at position -1 moves in the negative direction. It will be repaired at the first factory.

The limit of the first factory is 1, and it fixed 1 robot.

The limit of the second factory is 1, and it fixed 1 robot.

The total distance is $|2 - 1| + |(-2) - (-1)| = 2$. It can be shown that we cannot achieve a better total distance than 2.

Constraints:

- $1 \leq \text{robot.length}, \text{factory.length} \leq 100$
- $\text{factory}[j].\text{length} == 2$
- $-10^9 \leq \text{robot}[i], \text{position}_j \leq 10^9$
- $0 \leq \text{limit}_j \leq \text{robot.length}$
- The input will be generated such that it is always possible to repair every robot.

Python:

class Solution:

```
def minimumTotalDistance(self, robot: List[int], factory: List[List[int]]) -> int:
```

```
    robot.sort()
```

```
    factory.sort()
```

```
    n, m = len(robot), len(factory)
```

```
    INF = float('inf')
```

```
    dp = [[INF]*(m+1) for _ in range(n+1)]
```

```
    for j in range(m+1):
```

```
        dp[0][j] = 0
```

```
    for j in range(1, m+1):
```

```
        pos, limit = factory[j-1]
```

```
        for i in range(n+1):
```

```
            dp[i][j] = dp[i][j-1]
```

```
            dist = 0
```

```
            for k in range(1, min(limit, i)+1):
```

```
                dist += abs(robot[i-k] - pos)
```

```
dp[i][j] = min(dp[i][j], dp[i-k][j-1] + dist)
```

```
return dp[n][m]
```

JavaScript:

```
/**
 * @param {number[]} robot
 * @param {number[][]} factory
 * @return {number}
 */
// JavaScript

var minimumTotalDistance = function(robot, factory) {
    robot.sort((a, b) => a - b);
    factory.sort((a, b) => a[0] - b[0]);

    const m = robot.length;
    const n = factory.length;

    const dp = Array(m + 1).fill().map(() => Array(n + 1).fill(0));

    for (let i = 0; i < m; i++) {
        dp[i][n] = Infinity;
    }

    for (let j = n - 1; j >= 0; j--) {
        let prefix = 0;
        const qq = [[m, 0]];

        for (let i = m - 1; i >= 0; i--) {
            prefix += Math.abs(robot[i] - factory[j][0]);

            if (qq[0][0] > i + factory[j][1]) {
                qq.shift();
            }

            while (qq.length && qq[qq.length - 1][1] >= dp[i][j] + 1 - prefix) {
                qq.pop();
            }

            qq.push([i, dp[i][j] + 1 - prefix]);
            dp[i][j] = qq[0][1] + prefix;
        }
    }
}
```

```
    return dp[0][0];  
};
```

Java:

```
class Solution {  
    public long minimumTotalDistance(List<Integer> robot, int[][] factory) {  
        Collections.sort(robot);  
        Arrays.sort(factory, (a, b) -> a[0] - b[0]);  
  
        int n = robot.size();  
        int m = factory.length;  
  
        long INF = (long)1e18;  
        long[][] dp = new long[n + 1][m + 1];  
  
        for (int i = 0; i <= n; i++)  
            Arrays.fill(dp[i], INF);  
  
        dp[0][0] = 0;  
  
        for (int j = 1; j <= m; j++) {  
            int pos = factory[j - 1][0];  
            int limit = factory[j - 1][1];  
  
            for (int i = 0; i <= n; i++) {  
                dp[i][j] = dp[i][j - 1];  
  
                long dist = 0;  
                for (int k = 1; k <= limit && i - k >= 0; k++) {  
                    dist += Math.abs(robot.get(i - k) - pos);  
                    dp[i][j] = Math.min(dp[i][j], dp[i - k][j - 1] + dist);  
                }  
            }  
        }  
  
        return dp[n][m];  
    }  
}
```